Cisco Secure Workload User Education Guide

Introduction, Video Library, and Onboarding Runbook

Cisco Secure Workload User Education

2026-05-14

Table of Contents

Cisco Secure Workload User Education Guide

Purpose

This guide helps users understand **Cisco Secure Workload (CSW)**, use a curated video library for learning, and follow a practical onboarding runbook for discovery, deployment, application dependency mapping, policy modeling, and staged micro-segmentation enforcement.

Use this as a user-friendly starting point. It is intentionally written in practical language, not as a replacement for Cisco product documentation, release notes, or design validation.

---

1. What Cisco Secure Workload Really Is

Cisco Secure Workload is a workload visibility and micro-segmentation platform. It helps customers understand how applications communicate, convert that understanding into policy, and enforce least-privilege communication across servers, virtual machines, cloud workloads, and supported application environments.

In simple terms, CSW answers four practical questions:

1. **What do I have?**
   Inventory workloads, labels, operating systems, processes, and available metadata from agents, cloud connectors, CMDB systems, and other integrations.

2. **What talks to what?**
   Observe L3 / L4 flows and, where supported, process-level context so teams can understand real application dependencies instead of guessing.

3. **What should be allowed?**
   Use observed flows, labels, application groups, and policy modeling to recommend or build least-privilege segmentation policies.

4. **How do I enforce safely?**
   Move from visibility to monitor mode, then to staged enforcement, with rollback and operational validation.

CSW is not just a firewall rule tool. It is an application dependency mapping, workload labeling, policy modeling, and micro-segmentation platform. The value comes from combining telemetry, labels, policy simulation, and controlled enforcement.

---

2. What CSW Solves: Lateral Movement and Ransomware

Cisco Secure Workload exists to break the assumption that an attacker who gets inside the network can move freely. Most enterprise networks are still relatively flat at the workload layer — once one server is compromised through a phishing link, an exposed admin port, a vulnerable web app, or a stolen credential, the attacker can usually reach hundreds of other servers, file shares, and databases over the same internal network paths that legitimate applications use.

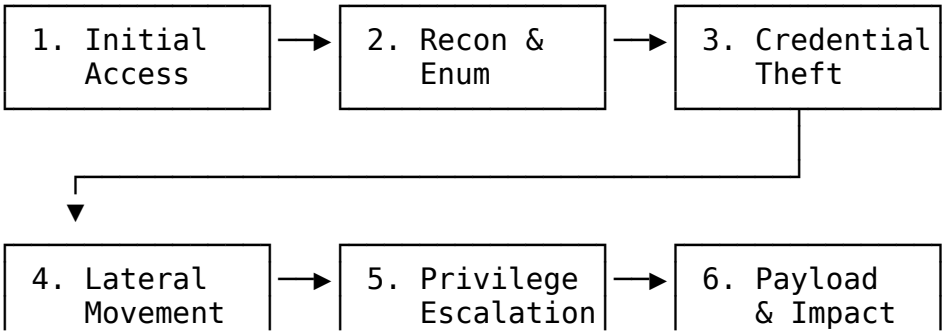
CSW reduces the **blast radius** of an intrusion by limiting each workload to **only the communication it actually needs**, based on real observed application behavior.

2.1 The Lateral Movement Problem

After initial access, modern attackers and most ransomware operators follow a predictable pattern:

1. **Land on one workload** — phishing, exposed RDP / SSH, exploited service, supply chain.
2. **Discover the internal network** — port scans, SMB enumeration, Active Directory reconnaissance.
3. **Steal credentials** — Mimikatz, LSASS dumps, Kerberoasting.
4. **Move laterally** — RDP, SMB, WinRM, WMI, PsExec, SSH, RPC, and other lateral admin tools.
5. **Escalate to high-value targets** — domain controllers, backup servers, file servers, databases, hypervisors.
6. **Stage and detonate the payload** — ransomware, data exfiltration, destruction.

Visually, the kill chain looks like this:



```
└──────────────┘    └──────────────┘    └──────────────┘

  * CSW intervenes between steps 4 and 5:
    least-privilege policy removes the
    workload-to-workload network paths
    these steps depend on. The attacker
    can still land on host 1, but cannot
    reach hosts 2..N from there.
```

Steps 2 through 5 **all depend on the network allowing workload-to-workload traffic that no business application actually requires**. That is exactly the layer CSW controls.

2.2 How CSW Stops or Slows Ransomware

Ransomware groups make money by encrypting many systems in a short time window. They depend on:

- Unrestricted **SMB / file share access** between workstations and servers.
- Open **RDP / WinRM / WMI / PsExec** paths between servers.
- Reachable **backup servers, file servers, and hypervisors** from compromised endpoints.
- Free **east-west traffic** that lets the malware fan out before defenders can react.

CSW directly attacks each of those preconditions:

Ransomware behavior	What CSW does about it
SMB / RDP / WinRM fan-out from a compromised workload	Allow only application-required ports between specific workload groups; deny lateral admin protocols by default.
Mass file-share encryption	Restrict file-server access to the specific workloads, users, and processes that genuinely need it.
Reaching backup or recovery systems	Place backup servers in their own scope; permit traffic only from backup agents, not from general workloads.
Reaching domain controllers or identity tier	Scope the identity / Active Directory tier so only required clients and admin jump hosts can reach it.
Hopping between application tiers	Enforce per-app and per-tier policy (web → app → database) so a compromised web tier cannot reach unrelated databases.
Spreading from dev / test / lab into prod	Hard-segment prod from non-prod so a non-prod compromise cannot pivot into prod.

Ransomware behavior	What CSW does about it
Long undetected dwell time	Surface every blocked or anomalous flow as evidence for SOC and incident response.

The result is that **ransomware that lands on one workload finds the network around it almost empty**: the protocols it needs to spread are blocked by policy, and every attempt is logged.

2.3 Why We Need It

- **Flat networks no longer match the threat model.** Perimeter firewalls do not stop an attacker who is already inside.
- **Identity-based and EDR controls are necessary but not sufficient.** They catch behavior on the host. CSW removes the network paths the attacker would use between hosts.
- **Crown-jewel applications need explicit protection.** Payments, claims, customer data, intellectual property, and backup infrastructure should not be reachable from a random user workstation or a low-tier dev server.
- **Compliance and audit demand it.** PCI, HIPAA, SOX, and most internal security frameworks expect documented segmentation between regulated and non-regulated systems.
- **It must not break applications.** CSW's discovery-first model (map dependencies → label workloads → model policy → enforce in stages) is what makes segmentation finally feasible in real enterprises.

In one sentence: **CSW exists so that when — not if — one workload is compromised, the attacker cannot reach the next one.**

---

3. Why Customers Care

Customers usually evaluate CSW because they need to reduce lateral movement risk without breaking applications. Traditional network segmentation often depends on VLANs, subnets, firewall zones, or static IP lists. That model becomes difficult when applications span data centers, public cloud, containers, and shared services.

CSW helps customers:

- Discover application dependencies before enforcing policy.
- Build policies based on labels and application intent, not only IP addresses.
- Protect high-value or vulnerable applications with tighter controls.
- Align policy with CMDB data, cloud tags, application ownership, and observed behavior.
- Roll out segmentation in phases, starting with visibility and policy modeling.
- Produce evidence for security, audit, and architecture teams.

---

4. Core CSW Concepts for Users

Concept	User-friendly explanation
Workload	A server, VM, cloud instance, container host, or supported compute asset being observed or protected.
Agent / enforcer	Software installed on supported workloads to collect telemetry and, when enabled, enforce policy.
Flow visibility	The observed network communication between workloads, usually source, destination, protocol, port, and time.
Process visibility	Mapping a network flow to the local process or service using it, where supported.
Labels	Metadata used to describe workloads, such as application, environment, role, owner, location, or risk.
Scope	A logical boundary for organizing workloads and policies.
Policy modeling	Previewing what would be allowed or blocked before enforcing.
Monitor mode	Observe and report policy behavior without blocking traffic.
Enforcement mode	Actively allow or deny traffic based on policy.
Connectors	Integrations that bring in context from systems such as ServiceNow, cloud providers, or other sources.
Edge appliance	A CSW virtual appliance used to run certain connectors, such as the ServiceNow connector.
Ingest	A pipeline or appliance pattern used where external telemetry ingestion is required. Validate design with Cisco for the customer use case.

5. CSW Video Library

Note: The screenshot provided showed visible **Watch here** labels but not the actual hyperlink targets. The table below creates the video-library structure and short descriptions. Replace `URL needed` with the real links when available.

#	Video topic	Short description	Link
1	CSW overview	Introduces what Cisco Secure Workload is, the business problem it solves, and how it supports application visibility and segmentation.	URL needed
2	Workload discovery	Shows how CSW identifies workloads, inventories assets, and starts building application context.	URL needed
3	Flow visibility	Explains L3 / L4 flow visibility and how teams use observed traffic to understand dependencies.	URL needed
4	Process visibility	Shows process-level context and why it matters for reducing overly broad port-based policies.	URL needed
5	Labels and scopes	Explains label strategy, scopes, and how labels are used to group workloads for policy.	URL needed
6	Application dependency mapping	Demonstrates how CSW visualizes application communication and dependency relationships.	URL needed
7	Policy	Shows how	URL needed

#	Video topic	Short description	Link
	recommendation	observed behavior can be converted into recommended segmentation policy.	
8	Policy modeling	Explains how to preview policy impact before moving to enforcement.	URL needed
9	Enforcement workflow	Demonstrates monitor mode, staged enforcement, blocked traffic, and rollback planning.	URL needed
10	Integrations and connectors	Covers ServiceNow, cloud connectors, identity / directory context, SIEM / SOAR, and other ecosystem integrations.	URL needed

Suggested Watch Order

1. Start with **CSW overview**.
2. Watch **Workload discovery**, **Flow visibility**, and **Process visibility** before talking about enforcement.
3. Watch **Labels and scopes** before policy design.
4. Watch **Application dependency mapping**, **Policy recommendation**, and **Policy modeling** before customer workshops.
5. Watch **Enforcement workflow** only after discovery and policy modeling are understood.
6. Watch **Integrations and connectors** when preparing for a POV or architecture review.

6. How to Explain CSW in a Customer Meeting

30-second version

Cisco Secure Workload helps customers discover how applications communicate and then safely enforce least-privilege segmentation policies around those applications. It starts with visibility and dependency mapping, uses labels and observed behavior to create policy, and supports staged enforcement so teams can reduce lateral movement risk without breaking applications.

2-minute version

Most enterprises want micro-segmentation, but they hesitate because they do not fully know what their applications depend on. CSW starts by collecting workload telemetry, flow visibility, process context, cloud metadata, and CMDB labels. That data helps application teams understand real dependencies. From there, teams can model policy, see what would be blocked, tune the rules, and move selected workloads from monitor mode to enforcement. CSW is especially useful when customers need consistent segmentation across data center and cloud environments.

Common positioning

- **For security:** Reduce lateral movement and contain high-risk applications.
- **For network teams:** Shift from static IP / subnet rules to workload-aware policy.
- **For application teams:** Discover dependencies before enforcement and avoid unexpected outages.
- **For cloud teams:** Use cloud tags and dynamic metadata so policy follows workloads.
- **For operations:** Start in monitor mode, model changes, and roll out enforcement gradually.

---

7. CSW Onboarding Runbook

Phase 0 - Confirm Scope and Outcomes

Goal: Agree on what the customer wants to prove.

Steps:

1. Identify the target customer environment: data center, public cloud, containers, or a mix.
2. Select 2-3 representative applications.
3. Pick at least one low-risk application and one higher-value application.
4. Define success criteria:
    - Flow visibility demonstrated.
    - Process visibility demonstrated.
    - Application dependency map reviewed by app owner.
    - Labels imported or created.
    - Policy modeled before enforcement.

- One controlled enforcement test completed.

Output: POV scope, application list, stakeholder list, and success criteria.

Phase 1 - Prepare the Tenant and Access

Goal: Prepare the CSW management environment and user access.

Steps:

1. Confirm CSW tenant or cluster access.
2. Create required administrator, operator, and viewer accounts.
3. Configure SSO / MFA / RBAC if in scope.
4. Confirm API access if automation or reporting is part of the POV.
5. Document who can create labels, scopes, policies, and enforcement changes.

Output: Access matrix, RBAC model, and tenant readiness confirmation.

Phase 2 - Build the Label Strategy

Goal: Establish labels before policy design.

Recommended starting labels:

Label type	Example values	Source
Application	`payments`, `claims`, `customer-portal`	CMDB, app owner, manual
Environment	`prod`, `non-prod`, `dev`, `test`	CMDB, cloud tag
Role	`web`, `app`, `db`, `middleware`	CMDB, observed behavior
Location	`dc1`, `dc2`, `aws`, `azure`	Cloud connector, manual
Owner	`team-a`, `team-b`	CMDB
Risk	`high`, `medium`, `low`	vulnerability scanner, security team

Steps:

1. Agree on a minimal label taxonomy.
2. Map sources for each label: ServiceNow, cloud tags, manual, API, or observed behavior.
3. Avoid creating too many labels before discovery. Start simple.
4. Validate that labels can support the first policy examples.

Output: Initial label dictionary and source-of-truth mapping.

Phase 3 - Deploy Agents and Connectors

Goal: Start collecting telemetry and context.

Steps:

1. Install CSW agents on selected Linux / Windows workloads.
2. Confirm agent health and visibility.
3. Deploy a CSW Edge virtual appliance if connector workloads are required.
4. Configure ServiceNow connector if CMDB labels are in scope.
5. Configure cloud connectors if public cloud inventory / tags are in scope.
6. Configure Ingest only if external flow or metadata ingestion is required and validated by Cisco design.

Output: Healthy agents, connector status, imported labels, and visible workload inventory.

Phase 4 - Discover Application Dependencies

Goal: Learn how the application actually communicates.

Steps:

1. Let workloads run through normal business transactions.
2. Capture L3 / L4 flow data.
3. Review process-level visibility where supported.
4. Build application dependency maps.
5. Review dependencies with application owners.
6. Identify unexpected flows, shared services, admin paths, and risky protocols.

Output: Dependency map, flow summary, process evidence, and app-owner review notes.

Phase 5 - Model Policy

Goal: Build policy without breaking the application.

Steps:

1. Use observed flows and labels to create recommended allow rules.
2. Group policy by application tiers where possible.
3. Identify required shared services such as DNS, NTP, authentication, logging, monitoring, backup, and patching.
4. Model the policy before enforcement.
5. Review predicted blocked flows with application and operations teams.

6. Tune policy until the expected business transactions are allowed.

Output: Proposed policy, modeled impact, predicted denies, and approval to test enforcement.

Phase 6 - Start in Monitor Mode

Goal: Validate policy behavior before blocking.

Steps:

1. Apply policy in monitor or non-blocking mode where supported.
2. Run application test transactions.
3. Review policy violations and unexpected flows.
4. Update labels or policy rules as needed.
5. Confirm application owners are comfortable with the modeled impact.

Output: Monitor-mode results, policy tuning notes, and enforcement readiness decision.

Phase 7 - Enforce in a Controlled Scope

Goal: Prove micro-segmentation safely.

Steps:

1. Select a small enforcement scope.
2. Schedule a change window.
3. Confirm rollback plan.
4. Move selected workloads to enforcement.
5. Test allowed traffic.
6. Test intentionally denied traffic.
7. Confirm logs, alarms, and reports show the expected result.
8. Monitor application health after enforcement.

Output: Enforcement proof, blocked-flow evidence, allowed-flow evidence, rollback plan, and operational signoff.

Phase 8 - Operationalize

Goal: Make CSW sustainable after the POV.

Steps:

1. Define ownership for labels, scopes, and policies.
2. Document policy change process.

3. Integrate logs / alarms with SIEM or SOAR if in scope.
4. Create recurring policy review cadence.
5. Build API or CI/CD automation where needed.
6. Define exception handling for unsupported workloads.
7. Expand from the initial application set to the next wave.

Output: Operating model, change process, exception process, and expansion roadmap.

---

8. Discovery Questions

Use these questions before sizing or designing a POV:

1. Which business application should be used for the first dependency map?
2. Which application owner can validate the map?
3. Which environments are in scope: prod, non-prod, dev, test?
4. Which platforms are in scope: VMware, bare metal, public cloud, containers?
5. Which operating systems are in scope?
6. Is ServiceNow the CMDB source of truth?
7. Are cloud tags clean enough to use as labels?
8. Is MFA / SSO required for CSW administrator access?
9. Are agents allowed on the selected workloads?
10. Is enforcement part of the POV, or visibility-only?
11. What is the rollback process for policy enforcement?
12. Which SIEM / SOAR tools should receive alarms or logs?
13. What vulnerability scanner or risk source should be used for high-risk labels?
14. Who approves policy changes?
15. What evidence does the customer need at the end of the POV?

---

9. POV Evidence Checklist

Evidence	Why it matters
Agent health screenshot	Proves telemetry source is active
Workload inventory screenshot	Proves assets are visible
Label inventory screenshot	Proves metadata is usable
ServiceNow connector status	Proves CMDB integration path
Cloud connector status	Proves cloud context ingestion
Flow table export	Proves L3 / L4 visibility
Process-level flow screenshot	Proves process context where supported
Application dependency map	Proves discovery value

Evidence	Why it matters
Policy recommendation	Proves policy can be derived from observed behavior
Policy modeling output	Proves change impact can be reviewed before enforcement
Monitor-mode policy result	Proves safe pre-enforcement workflow
Enforcement blocked-flow test	Proves actual segmentation control
Allowed business transaction test	Proves enforcement did not break required traffic
Audit log	Proves administrative traceability
API output	Proves automation path
SIEM event	Proves operational integration

10. Common Pitfalls

Pitfall	How to avoid it
Starting with enforcement too early	Start with discovery, labels, and modeling before blocking traffic.
Using too many labels too soon	Start with application, environment, role, location, owner, and risk. Expand later.
Skipping application owner validation	Dependency maps must be reviewed by the app owner before policy.
Ignoring shared services	Always account for DNS, NTP, identity, logging, monitoring, backup, and patching.
Treating cloud and data center separately	Use consistent labels across both so policy is portable.
Forgetting rollback	Every enforcement test needs a rollback plan and owner.
Overclaiming integration support	Confirm exact platform and version support with current Cisco documentation.

11. Simple CSW Talk Track

Use this flow in customer conversations:

1. **Start with the risk:** Attackers move laterally after initial compromise. Flat networks make that easier.

2. **Explain the blocker:** Customers hesitate to segment because they do not know every application dependency.
3. **Position CSW:** CSW discovers dependencies first, then models policy, then enforces gradually.
4. **Make it practical:** We start with a few representative apps, import labels, review flows, model policy, and only then enforce.
5. **Close with outcome:** The customer gets application maps, policy recommendations, enforcement proof, and an expansion roadmap.

---

12. Open Items Before Publishing Final Version

1. Replace all `URL needed` placeholders in the video library with real video links.
2. Confirm whether this repo should be public, private, or internal-only.
3. Confirm whether customer-specific examples should be removed or kept generic.
4. Add official Cisco documentation links for the final user education version.
5. Validate current release-specific support matrices before presenting platform coverage.